

06 Forms and Inputs

Form attributes · Input types · Labels · Validation

The skeleton of a form

```
<form action="/signup" method="POST">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required placeholder="you@mail.com">
  <button type="submit">Sign Up</button>
</form>
```

Attribute	Purpose
<code>action</code>	The URL to which the data is sent.
<code>method</code>	<code>GET</code> places the data in the URL, which suits search and filter pages that should be bookmarkable. <code>POST</code> places it in the request body and is used for logins, signups and any sensitive data.
<code>name</code>	The most important attribute of a field. The server receives the value under this key. A field without a <code>name</code> is not submitted at all.
<code>id</code>	Used by <code><label for></code> , CSS and JavaScript. It is never sent to the server.
<code>enctype</code>	Must be set to <code>multipart/form-data</code> whenever the form uploads a file.

The rule about labels

Every input needs a label, either through `<label for="id">` or by wrapping the input inside the label. Screen readers then announce the purpose of the field, and clicking the label moves focus into the input, which also creates a larger tap target on mobile devices.

A PLACEHOLDER IS NOT A LABEL

The `placeholder` disappears the moment typing begins, has low contrast and is not announced reliably by every screen reader. Using it in place of a label is an accessibility failure. Its only purpose is to show an example of the expected format.

Input types

Type	Usage
<code>text</code> , <code>email</code> , <code>password</code> , <code>url</code> , <code>tel</code>	Text entry. <code>email</code> and <code>url</code> provide built-in validation, and mobile keyboards adapt to the type.
<code>number</code> , <code>range</code>	Numeric entry, used together with <code>min</code> , <code>max</code> and <code>step</code> .
<code>radio</code>	One choice out of a group. All options must share the same <code>name</code> for them to become mutually exclusive.
<code>checkbox</code>	Zero or more choices. Only a checked box is submitted, and it sends its <code>value</code> .
<code>date</code> , <code>time</code> , <code>datetime-local</code> , <code>color</code>	Native pickers provided by the browser.
<code>file</code>	File upload, refined with <code>accept</code> and <code>multiple</code> . The form must set <code>enctype</code> .
<code>hidden</code>	Not visible to the user but still submitted, typically a record identifier.
<code>submit</code> , <code>reset</code>	Form actions. Note that a <code><button></code> also defaults to <code>type="submit"</code> .

Other form elements

- `<select>` with `<option value="...">`, grouped by `<optgroup>` and made multi-choice with `multiple`.
- `<textarea rows cols>`, whose value is the **content between the tags** rather than a `value` attribute.
- `<fieldset>` with `<legend>` to group related fields, which is especially valuable for radio groups.
- `<datalist>` to offer suggestions while still allowing free text.

Built-in validation

- Available attributes include `required`, `minlength`, `maxlength`, `min`, `max`, `step` and `pattern="[0-9]{10}"`, along with the checks implied

by the input type.

- Adding `novalidate` to the form disables browser validation entirely.
- **Client-side validation is never a security measure.** A user can remove these attributes through developer tools or call the endpoint directly, so the server must validate everything again.

DISABLED VERSUS READONLY

A `disabled` field is greyed out, cannot receive focus and **is not submitted**. A `readonly` field cannot be edited but can be focused and copied, and **is submitted**. When a pre-filled value must reach the server, `readonly` is the correct choice.

QUICK RECAP

- The `name` attribute decides what reaches the server; `id` only connects the label.
- Radio buttons form a group through a shared `name`.
- Browser validation improves usability; server validation provides the actual protection.